

Ödev 14: Log Analizi

Bir log'u stream boru hatlarıyla analiz edin, `map` metodunun gidemeyeceği yere bir gatherer ile uzanın, ve paralel veri yarışını olurken izlemek için yeniden üretin.

Veri

```
List.of(
    "INFO auth 120",
    "ERROR auth 450",
    "INFO billing 80",
    "WARN billing 900",
    "ERROR auth 380",
    "INFO search 60",
    "ERROR billing 1200",
    "INFO auth 95",
    "WARN search 700",
    "INFO billing 110");
```

Her satırı `record Entry(String level, String service, int millis)` içine ayırıştırın.

Birinci Bölüm: Gruplama

`LogAnalysis.java` dosyasını klasik biçimde oluşturun. Şunların hepsini **tek** stream boru hattıyla üretin:

- Servis başına girdi sayısı
- Servis başına ortalama yanıt süresi
- Servis başına görülen farklı seviyeler, sıralı
- Girdilerin yavaş (500ms üstü) ve hızlı olarak iki gruba ayrılması
- Servis başına en yavaş süre

Gereksinimler:

- Gruplanmış her sonuç **öngörülebilir** bir anahtar sırasında yazdırılmalı. Hangi harita tipini kullandığınızı ve varsayılanın neden kabul edilemez olduğunu bir yorumda söyleyin.
- En az üçü için ikinci bir geçiş değil, aşağı akış collector'ı kullanın.
- İki gruba ayırma için `partitioningBy` kullanın ve bir boolean üzerindeki `groupingBy` seçeneğinin vermediği neyi garanti ettiğini bir yorumda yazın.
- En yavaş süre için önce düz bir `toMap` deneyin ve ne olduğunu kaydedin, sonra düzeltin.

İkinci Bölüm: Gatherer'lar

Yalnızca süreleri çıkarın, sonra şunları üretin:

1. Üçerli gruplar
2. Ardışık her okuma çifti arasındaki fark
3. Yürüyen toplam

Soru

Bu üçünün **hiçbirinin** neden `map` ile yazılamayacağını kendi kelimelerinizle açıklayan bir yorum yazın. Cevabınız üçünü de ele almalı, ve farklı sebeplerle aynı testte kalıyorlar. `map` metodunun uyduğu kuralı isimlendirin ve her birinin hangi kısmını çiğnediğini söyleyin.

Üçüncü Bölüm: Yarıştırın

Paralel bir stream'den düz bir `ArrayList` içine 50.000 tam sayı ekleyin. Bunu **üç kez** yapın ve her çalıştırmada ortaya çıkan boyutu yazdırın.

Bir try/catch ile koruyun, çünkü eleman kaybetmek tek başarısızlık biçimi değil.

Sonra doğru sayımı iki biçimde üretin: `.toList()` ve `.count()` ile.

Açıklayın

Şunları kapsayan bir yorum yazın:

1. **Mekanizma.** `ArrayList.add` birden fazla adımdır. Adımları isimlendirin ve araya giren iki iş parçacığının birbirine ne yapabileceğini söyleyin.
2. **collect neden bağışık.** Kaplarla farklı olarak ne yapıyor?
3. **Neden synchronized eklemek yanlış çözüm,** doğru olmasına rağmen.

Dördüncü Bölüm: Tembellik ve Tek Kullanım

İçinde `peek` olan ama terminal işlemi olmayan bir boru hattı kurun ve hiçbir şeyin çalışmadığını gösterin.

`findFirst` ekleyin ve peek çıktısından on girdinin hepsini incelemek yerine **erken durduğunu** gösterin.

Sonra aynı stream değişkeninde ikinci bir terminal işlem çağırın, ne olduğunu yakalayın ve mesajı yazdırın. Aynı cevabı kaynaktan doğru biçimde alarak bitirin.

Kabul Kriterleri

- ☐ `java LogAnalysis.java` ile çalışıyor
- ☐ Birinci bölümün beş sonucu da tek boru hattından geliyor
- ☐ Anahtar sırası öngörülebilir, harita tipini gerekçelendiren bir yorumla
- ☐ En az üç aşağı akış collector'ı kullanılıyor

- ☐ Bir yorum `partitioningBy` metodunun neyi garanti ettiğini belirtiyor
- ☐ `toMap` tekrarlayan anahtar hatası tetikleniyor ve sonra düzeltiliyor
- ☐ Üç gatherer işleminin hepsi doğru çıktı üretiyor
- ☐ Yorum `map` metodunun üçünü de neden yapamayacağını açıklıyor
- ☐ Üç paralel çalıştırma boyutlarını yazdırıyor, fırlatmaya karşı korunmuş
- ☐ Doğru sayım `.toList()` ve `.count()` ile gösteriliyor
- ☐ Yarış açıklaması mekanizmayı, `collect` neden güvenli ve kilit neden yanlış çözüm olduğunu kapsıyor
- ☐ peek çıktısı hem tembelliği hem kısa devreyi kanıtlıyor
- ☐ Stream tekrar kullanımı yakalanıyor ve mesajı yazdırılıyor

İleri Seviye

Üçüncü bölümdeki paralel stream'i `Gatherers.mapConcurrent(4, ...)` ile değiştirin.

Sonra ikisini karşılaştıran bir yorum yazın. Hangisi eşzamanlılığını sınırlıyor? Hangisi sırayı koruyor? Bir ağ servisi çağıran iş için hangisini kullanırdınız ve Modül 14 diğerinin orada neden kötü fikir olduğunu söylüyor?

İpucu, farklarınız bir eleman fazla çıkıyorsa

On eleman üzerinde `windowSliding(2)` on değil dokuz pencere verir. On okuma arasında dokuz boşluk vardır. On aldıysanız pencereleme mi yoksa eşleme mi yaptığınızı kontrol edin.